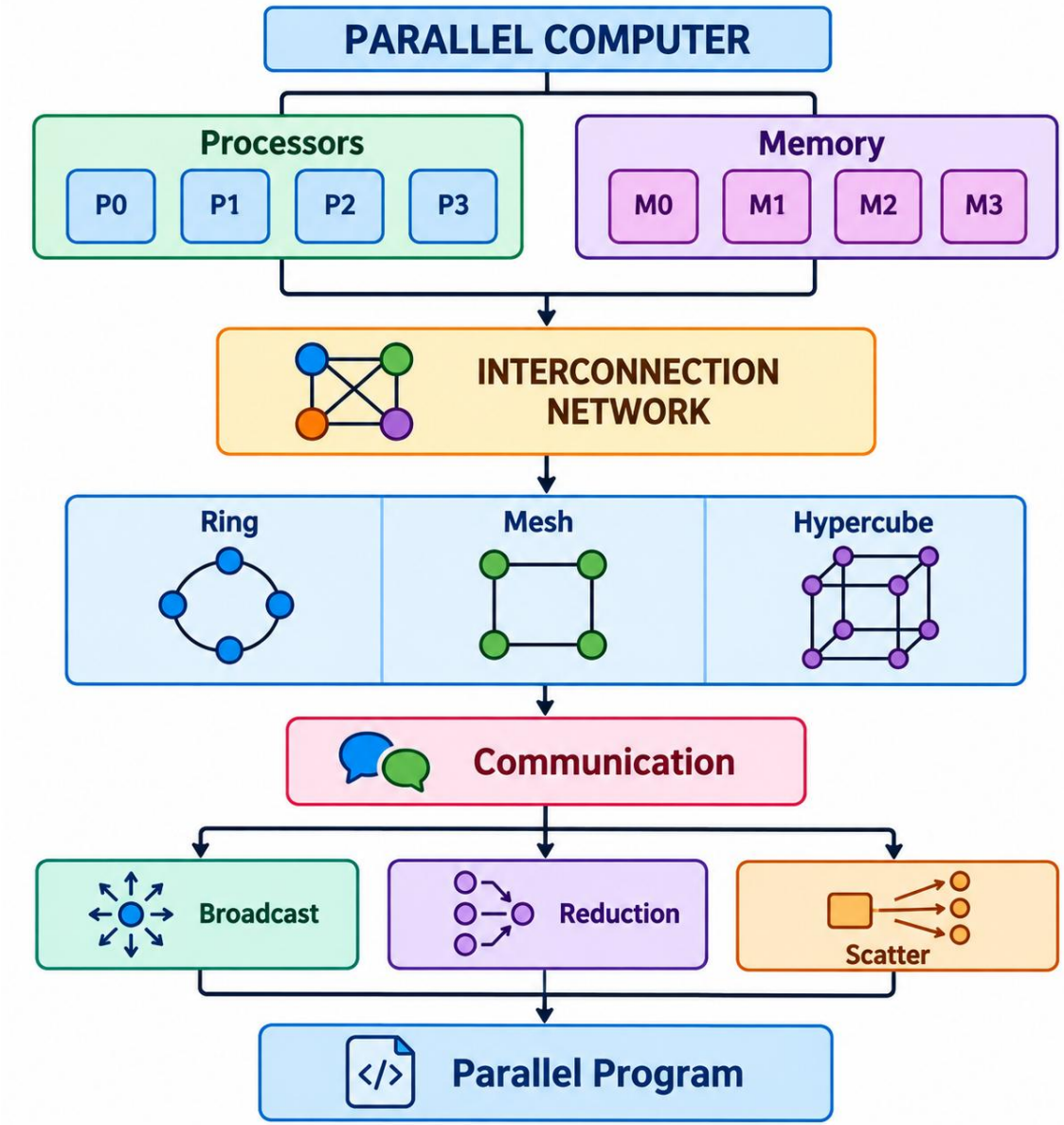
IS5103 – High Performance Computing

Interconnection Networks & Basic Communication Operations

T. Kayalvizhy | Faculty of Computing, SUSL

Interconnection Networks & Basic Communication Operations

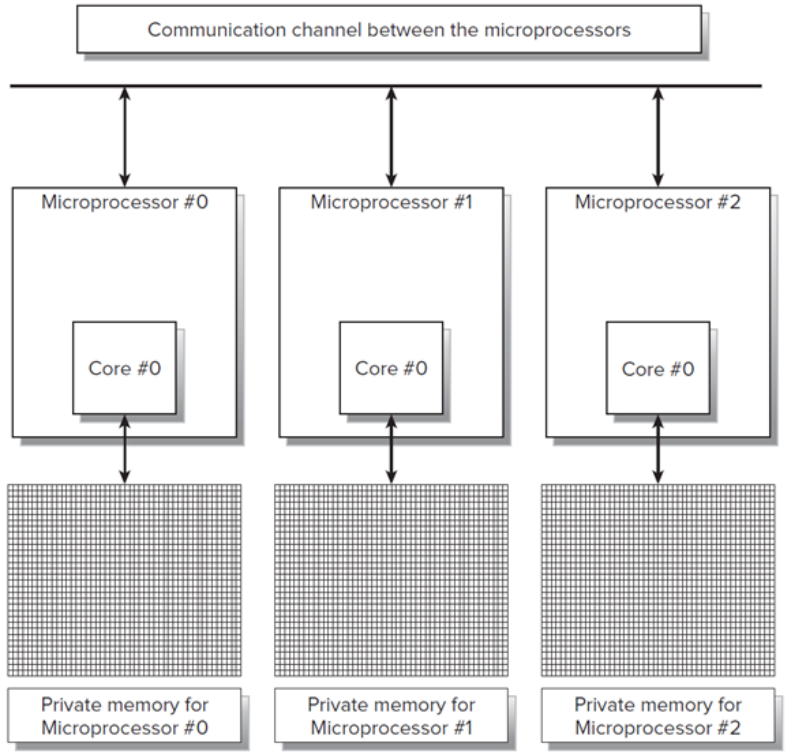


What Is a Parallel Computer?

- A parallel computer contains multiple processing elements that can work on different parts of a problem at the same time.
- Example: 8 images can be processed by 8 processors instead of one processor processing all 8 sequentially.
- Parallelism improves throughput and can reduce execution time when the workload can be divided effectively.
- But independent computation is not enough: processors often need to exchange data, results, or control information.

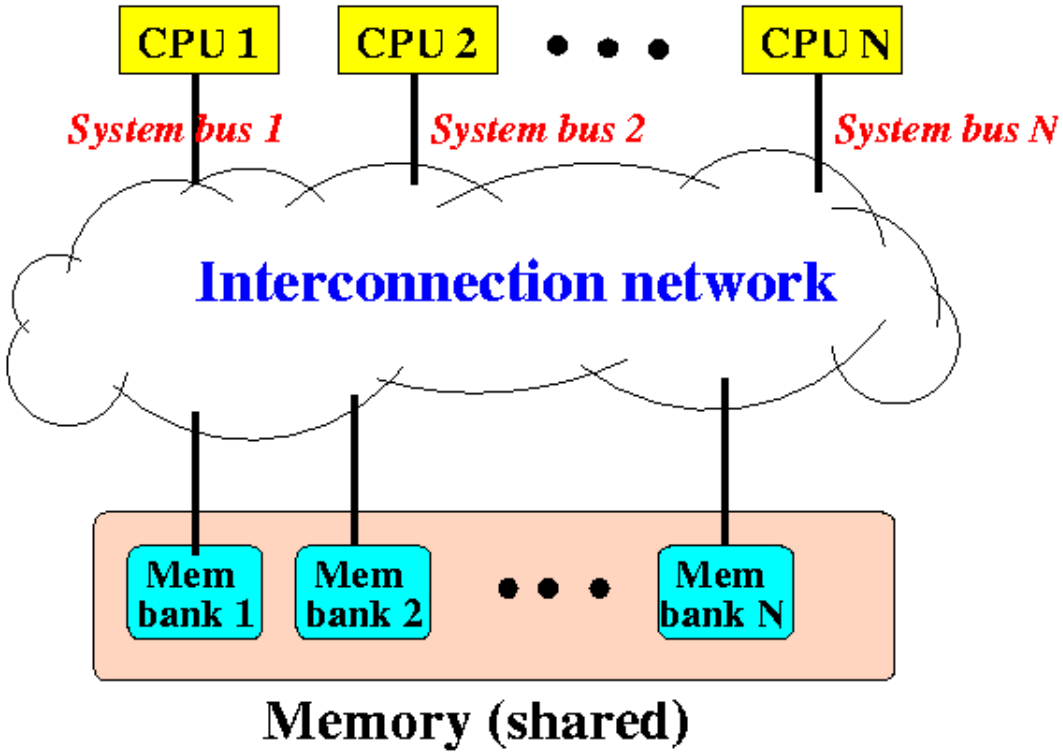
The Fundamental Communication Problem

- Suppose we have processors P0, P1, P2, and P3.
- P0 calculates a value, but P3 needs that value.
- The processors therefore require a communication path.
- This leads to the central question of the topic:
- **How are processors and memory connected, and how does data travel between them?**



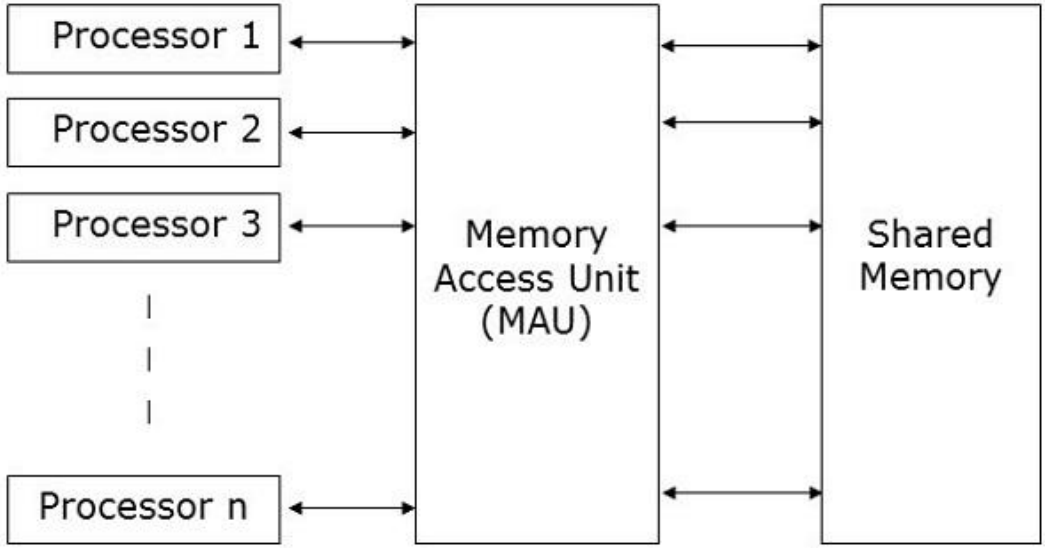
From Processor to Network

- The communication mechanism is the interconnection network.
- The network provides mechanisms for data transfer between processing nodes or between processors and memory modules.
- Typical networks are built using links and, in many designs, switches.



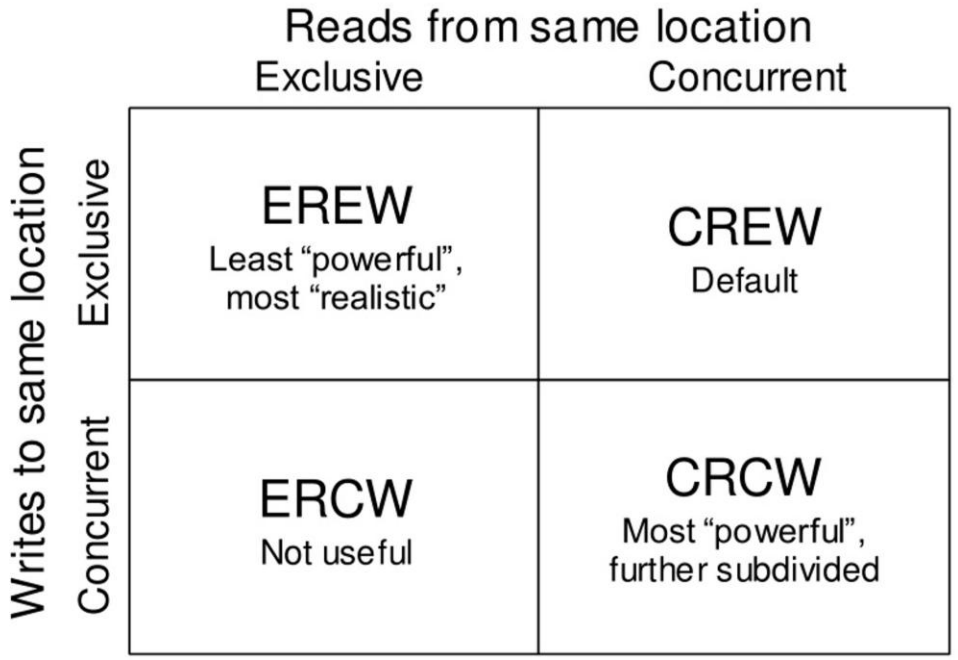
Parallel Random Access Machine (PRAM)

- A set of similar-type processors
- It consists of p processors and a shared global memory.
- All processors use the same address space.
- Processors can communicate among themselves through the shared memory only.
- Processors may execute different instructions in a cycle while sharing a common clock.
- A MAU connects the processors with the single shared memory.



PRAM Memory Access Models

- **EREW** — Exclusive Read, Exclusive Write: no concurrent reads or writes to a memory location.
- **CREW** — Concurrent Read, Exclusive Write: multiple processors may read; writes are serialized.
- **ERCW** — Exclusive Read, Concurrent Write: reads are serialized; concurrent writes are allowed with an arbitration rule.
- **CRCW** — Concurrent Read, Concurrent Write: both multiple reads and writes are allowed.
- Concurrent writes require a rule/protocol such as common, arbitrary, priority, or sum.



What Is an Interconnection Network?

- An interconnection network provides a mechanism for transferring data among processing nodes and/or between processors and memory modules.
- A simple black-box view has inputs and outputs.
- Links carry information.
- Switches can connect inputs to outputs and support routing, buffering, and multicast.
- Network interfaces connect a processing node to the network.

Good IPC = Low Latency + High Bandwidth + Low Communication Cost + Good Scalability

Main Building Blocks

Links

- Physical communication paths.
- Can be implemented using electrical or optical media.
- Carry bits/data between connected components.
- More links can improve connectivity but may increase cost.

Switches & Network Interface

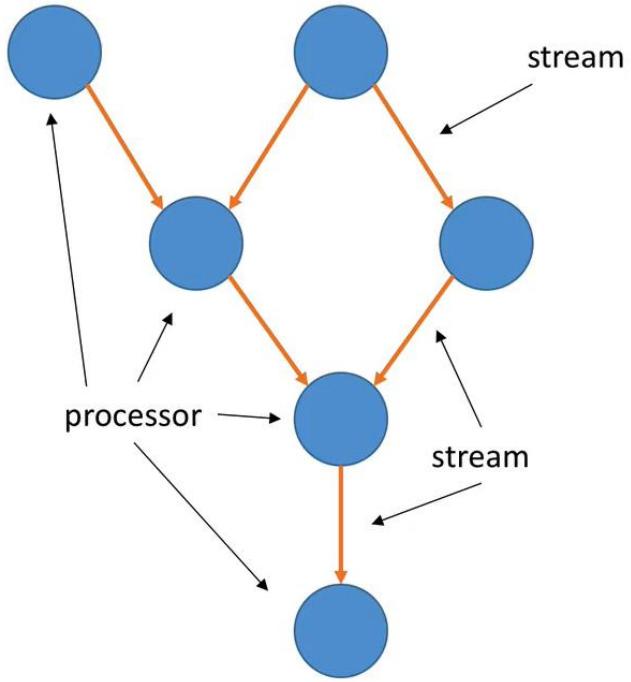
- Switch: maps input ports to output ports.
- May buffer data when an output port is busy.
- Supports routing and possibly multicast.
- The network interface handles functions such as packetization, routing information, buffering, and error checking.

Static vs Dynamic Networks

- **Static (direct) network:** fixed point-to-point communication links connect processing nodes.
- **Dynamic (indirect) network:** switches dynamically connect communication paths.
- Static networks are commonly described directly in terms of processor-to-processor topology.
- Dynamic networks are commonly described using switching elements and their connections.

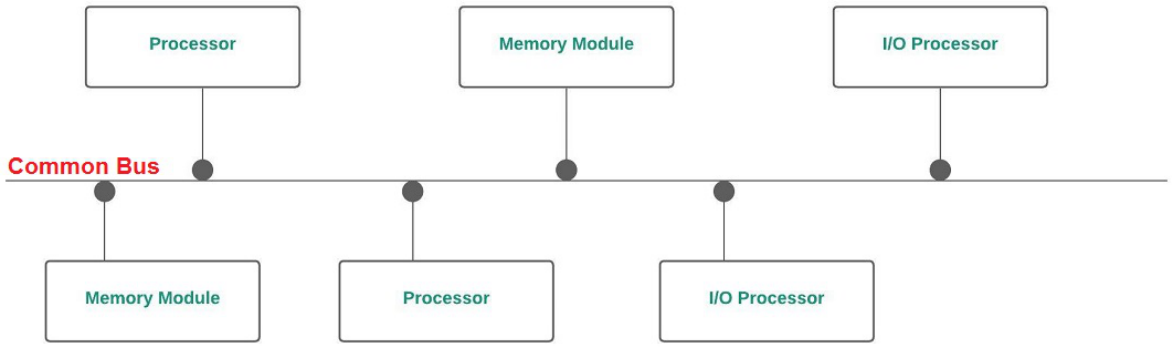
What Is Network Topology?

- Topology is the arrangement of nodes, links, and other network elements.
- Physical topology describes physical placement and connections.
- Logical topology describes how data flows, which may differ from physical placement.
- Different topologies trade off cost, latency, bandwidth, connectivity, and scalability.



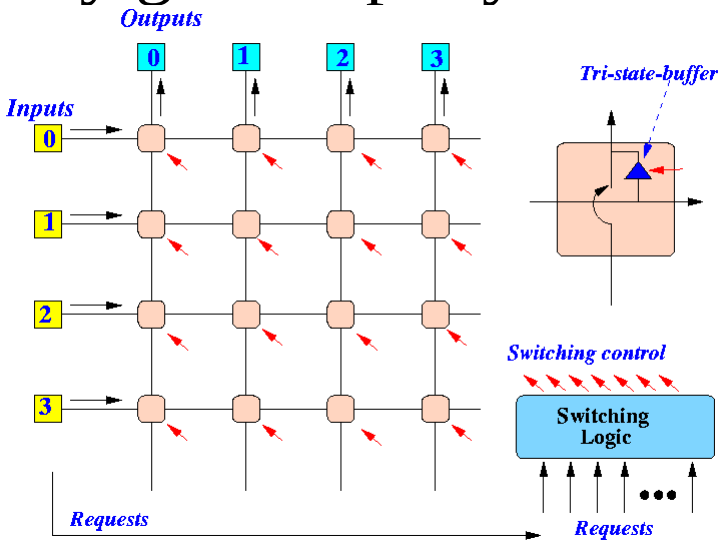
Bus-Based Network

- All connected devices share a common communication medium.
- Simple and relatively low-cost.
- A message can be seen by devices attached to the shared bus, with access controlled by hardware/protocol.
- Main weakness: many processors competing for one shared resource create contention.
- Analogy: one road shared by many vehicles.



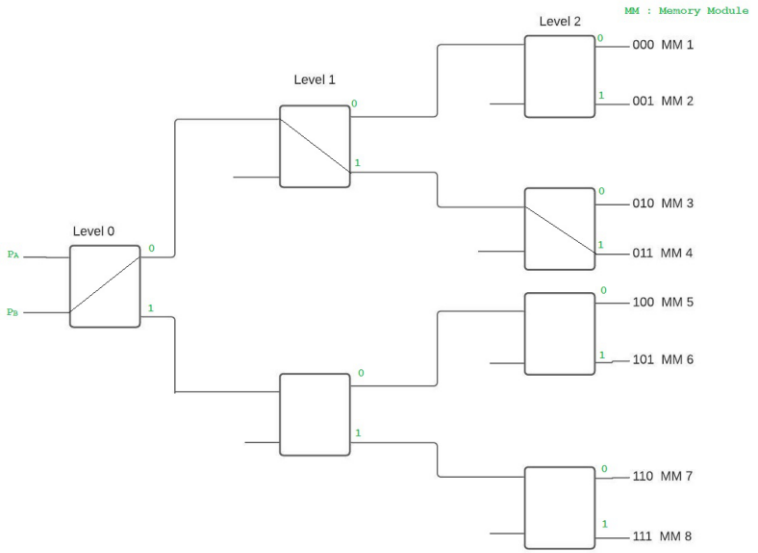
Crossbar Network

- A crossbar provides switch points between sources and destinations.
- Different source-destination pairs can communicate simultaneously when they do not request the same resource.
- It provides high connectivity and can be non-blocking except for conflicts involving the same destination/resource.
- Main disadvantage: hardware cost and complexity grow rapidly as the system scales.



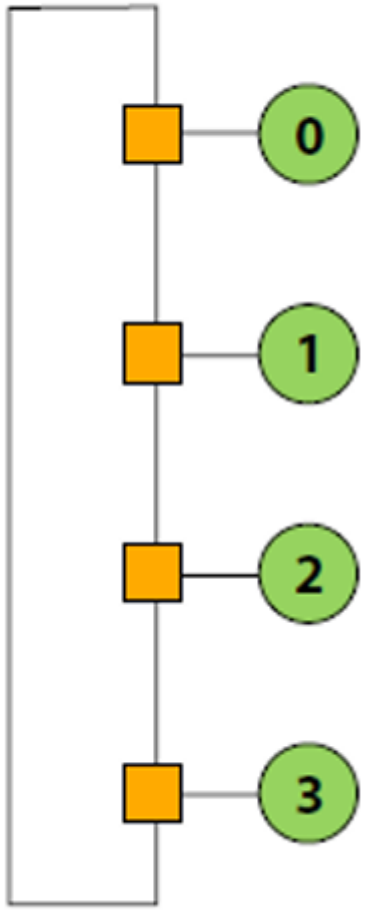
Multistage Interconnection Networks

- Use several stages of smaller switches rather than one large crossbar.
- Provides a compromise between the simplicity of a bus and the connectivity of a crossbar.
- Omega is an example of a multistage network.
- Messages pass through multiple switching stages.
- Some multistage networks are blocking: two messages may compete for the same internal path.



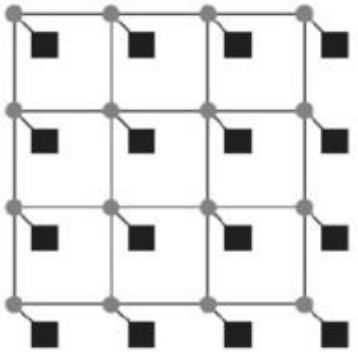
Ring Topology

- Each processing node is connected to two neighboring nodes in basic ring.
- Communication can travel around the ring in either direction depending on the routing strategy.
- Simple and low-degree.
- Longer paths can occur as the number of nodes grows.
- Ring algorithms are useful for broadcast, all-to-all, and other collective operations.



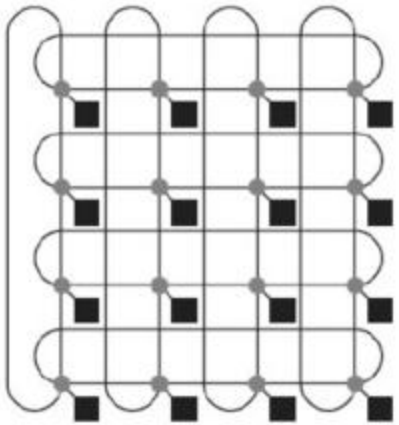
Mesh Topology

- Nodes are arranged in rows and columns.
- A 2D mesh connects a node to nearby horizontal and vertical neighbors.
- Example: a 4×4 mesh contains 16 nodes.
- Communication generally proceeds through multiple hops.
- Mesh structures are easy to visualize and map to grid-like workloads.



Torus / Wraparound Mesh

- A wraparound mesh adds connections between opposite boundaries.
- The left edge can connect to the right edge; the top edge can connect to the bottom edge.
- This reduces edge effects and can shorten some communication paths.
- The resulting structure is commonly called a torus.



Hypercube: Start Small

A hypercube is built recursively.

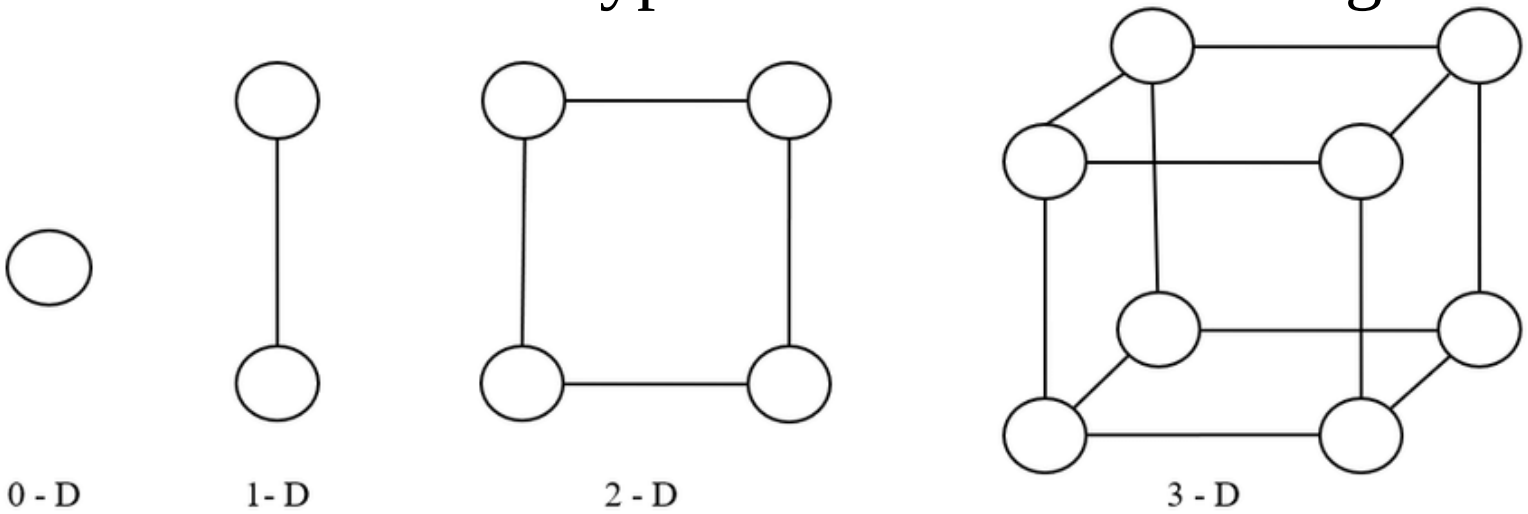
1-dimensional hypercube: 2 nodes.

2-dimensional hypercube: 4 nodes.

3-dimensional hypercube: 8 nodes.

In general, a d-dimensional hypercube has p = 2^d nodes.

Each node in a d-dimensional hypercube has d direct neighbors.



Hypercube Node Labels

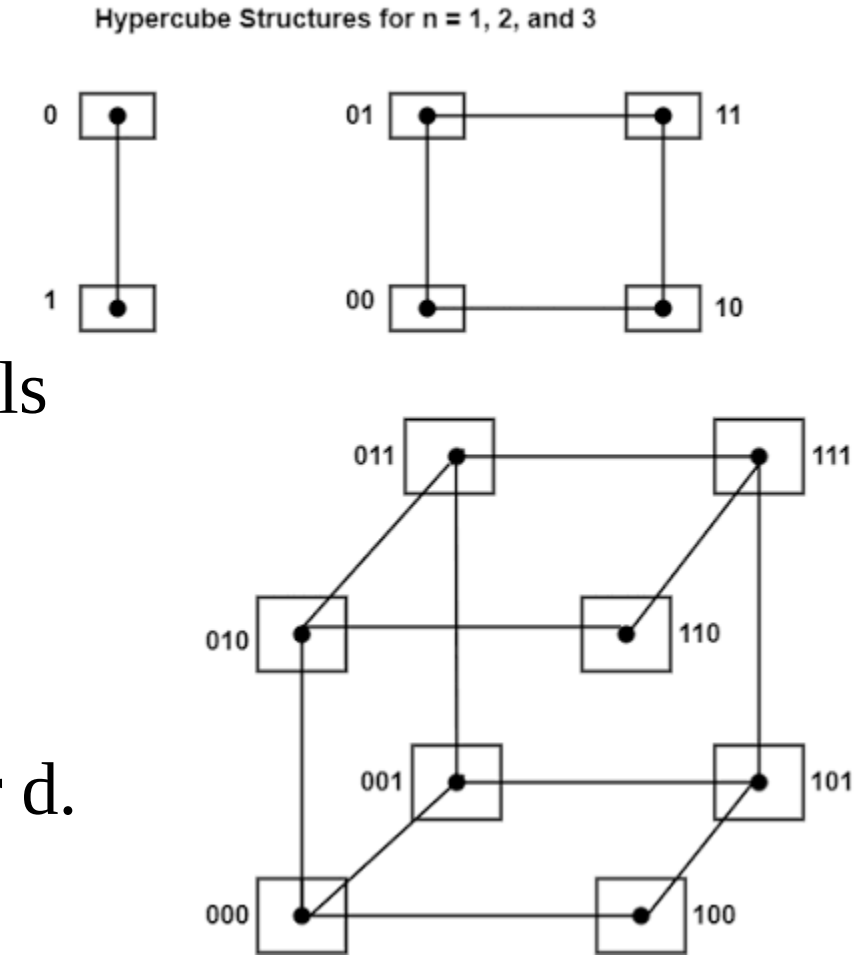
Nodes can be labeled using d-bit binary strings.

Two nodes are directly connected when their labels differ in exactly one bit.

The shortest-path distance between two labels equals the number of differing bits.

Example: 000 and 111 differ in 3 positions, so their distance is 3.

Therefore, a d-dimensional hypercube has diameter d.



Interconnection Topology: Performance Comparison

Topology	Number of Links	Distance	Diameter	Bisection Width	Main Advantage
Bus	**1 shared medium**	Variable	**N−1**	**1**	Simple and low cost
Ring	**N**	**N/2**	**⌊N/2⌋**	**2**	Simple and predictable
Star	**N−1**	**2**	**2**	**1**	Simple management
Mesh (2D)	**$2N - 2\sqrt{N}$**	**$\approx 2\sqrt{N}/3$**	**$2(\sqrt{N}-1)$**	**$\sqrt{N}$**	High connectivity and fault tolerance
Tree	**N−1**	Depends on tree	Depends on height	**1**	Scalable hierarchical structure
Hypercube	**$N \log_2 N / 2$**	**$\log_2 N$**	**$\log_2 N$**	**N/2**	Short paths and excellent scalability

Topology Selection: What Should We Optimize?

Requirement	Preferred Characteristic
Low latency	**Small diameter**
High bandwidth	**Multiple/high-capacity links**
Low cost	**Fewer links / simpler structure**
Large HPC system	**High scalability**
Reliable system	**High path redundancy**
Heavy communication	**Low congestion + high connectivity**

Inter-Process Communication: Performance Evaluation

Evaluation Metric	Meaning	Why It Matters in HPC
Communication Latency	Time required to initiate and complete a message transfer	Lower latency → faster communication
Bandwidth	Amount of data transferred per second	Higher bandwidth → better for large messages
Communication Cost	Time/overhead spent communicating instead of computing	Lower cost → better parallel efficiency
Average Message Latency	Average time taken for messages to reach the destination	Important when processes exchange many small messages
Number of Communication Links	Physical/logical connections between processing nodes	More suitable links can reduce congestion
Scalability	How communication performance changes as processes/nodes increase	Important for large parallel systems
Contention	Competition between processes for communication resources	High contention reduces performance
Throughput	Total amount of data/messages successfully transferred per unit time	Higher throughput → better system utilization

Key Network Topology Metrics

Metric	Definition	Why It Matters
Distance & Diameter	**Distance:** the shortest path between two nodes. **Diameter:** the largest shortest-path distance in the network.	Smaller diameter → fewer hops and lower worst-case communication delay.
Arc Connectivity	Minimum number of **links/edges** that must be removed to disconnect the network.	Higher connectivity → more alternative paths and better **fault tolerance**.
Bisection Width	Minimum number of links that must be cut to divide the network into **two equal halves**.	Higher bisection width → better communication capacity between network halves.

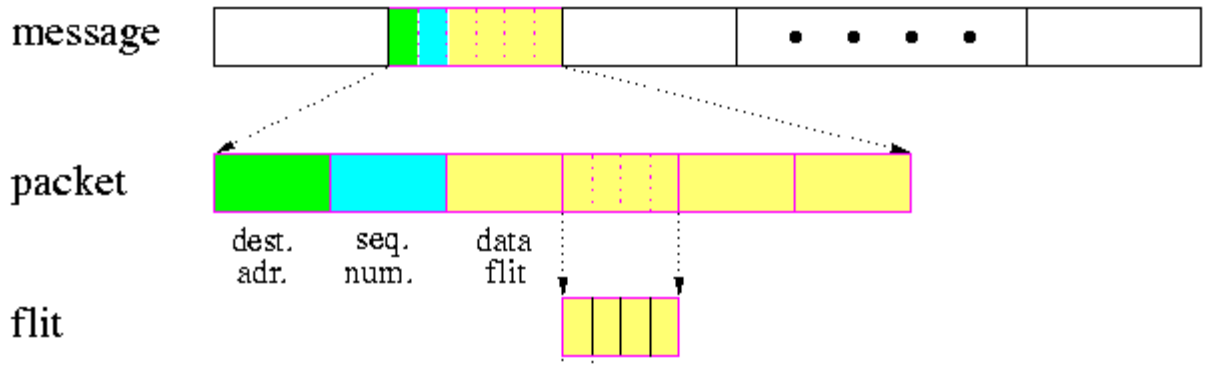
Communication Granularity

Message: a logical unit of transfer between communication endpoints such as cores or memory systems.

Packet: a unit of transfer for the network; a message may be divided into packets.

Flit (flow control digit): a smaller unit used for flow control; packets may be divided into flits.

Conceptual hierarchy: Message → Packet → Flit.



Store-and-Forward Routing

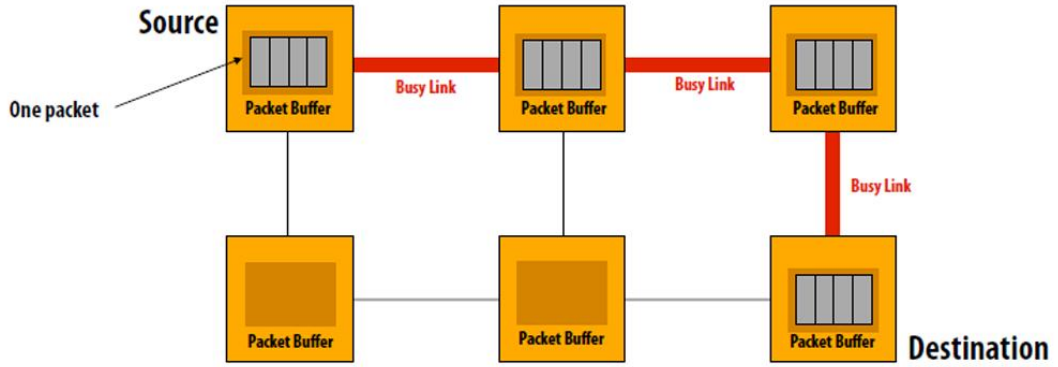
A message travels through multiple links/nodes.

Each intermediate node receives and stores the entire message.

Only after the complete message is received does the intermediate node forward it.

Advantage: conceptually simple.

Disadvantage: intermediate storage and repeated waiting can increase latency.



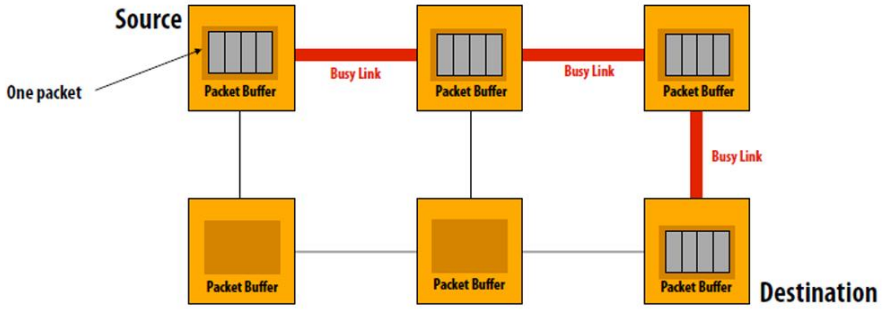
Cut-Through Routing

Messages are divided into smaller units such as flits.

A path is established, and the flits follow the path in sequence.

An intermediate node does not need to wait for the complete message before forwarding the incoming stream.

This can reduce latency and buffering requirements compared with store-and-forward.



Store-and-Forward vs Cut-Through

Store-and-Forward	Cut-Through
Receive entire message.	Message divided into flits.
Store at an intermediate node.	Path established.
Then forward.	Flits move sequentially.
More intermediate buffering.	Less need to store the whole message.
Latency grows with intermediate storage/forwarding.	Can reduce communication latency.

- Store-and-forward solution from the previous slide: 3 hops x 4 units of time to transmit a packet over a single link = 12 units of time
- Cut-through solution: 3 steps of latency for the head of the packet to get to the destination + 3 units of time for the rest of packet = 6 units of time

Communication Latency Components

Startup time (ts): time to prepare a message for transmission, add control information, execute routing setup, etc.

Per-hop time (th): time associated with the header moving between directly connected nodes.

Per-word transfer time (tw): time required to transfer one word over a link; if bandwidth is r words/second, tw = 1/r.

$$T = ts + tw \times m$$

ts = startup time

tw = per-word transfer time

m = message size in words

Store-and-Forward Routing

- A message traversing multiple hops is completely received at an intermediate hop before being forwarded to the next hop.
- The total communication cost for a message of size m words to traverse l communication links is

$$t_{comm} = t_s + (mt_w + t_h)l$$

- In most platforms, t_h is small, and the above expression can be approximated by

$$t_{comm} = t_s + mlt_w$$

Cut-Through Routing

- The total communication time for cut-through routing is approximated by

$$t_{comm} = t_s + t_h l + t_w m$$

- In this expression, t_h is typically smaller than t_s and t_w. For this reason, the second term in the RHS does not show, particularly when m is large.
- Furthermore, it is often not possible to control routing and placement of tasks.
- For these reasons, we can approximate the cost of message transfer by

$$t_{comm} = t_s + t_w m.$$

Collective Communication

Parallel programs often require structured communication among many processes.

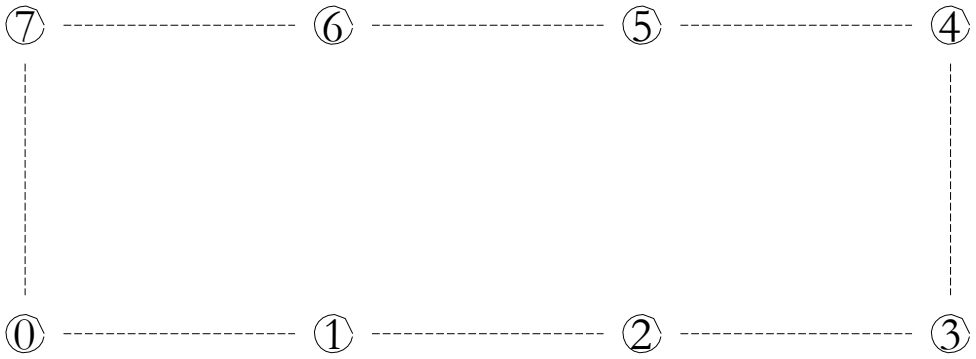
These patterns are called collective communication operations.

The source material covers broadcast, reduction, all-to-all broadcast, all-to-all reduction, all-reduce, prefix sum, scatter, gather and all-to-all personalized.

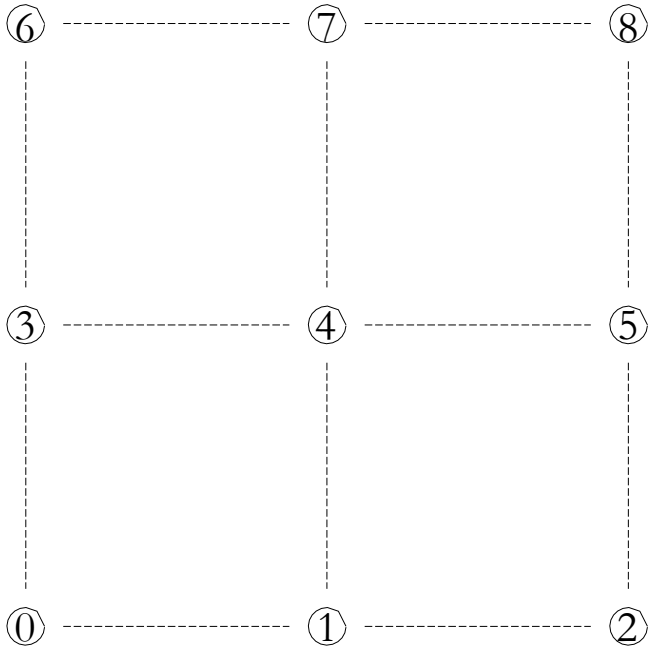
MPI provides collective functions corresponding to these patterns.

Topologies

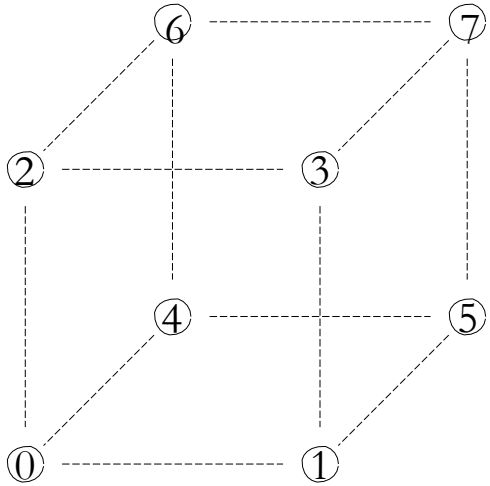
RING



2D-MESH



HYPERCUBE



Linear model of communication overhead

- Point-to-point message takes time with cut-through routing

$$t_s + t_w m$$

- t_s is the startup time
- t_w is the per-word transfer time (inverse bandwidth)
- m is the message size in # words

- Assuming bi-directional links
- Each node can send and receive simultaneously
- **Contention** if link is used by more than one message

One-to-all broadcast



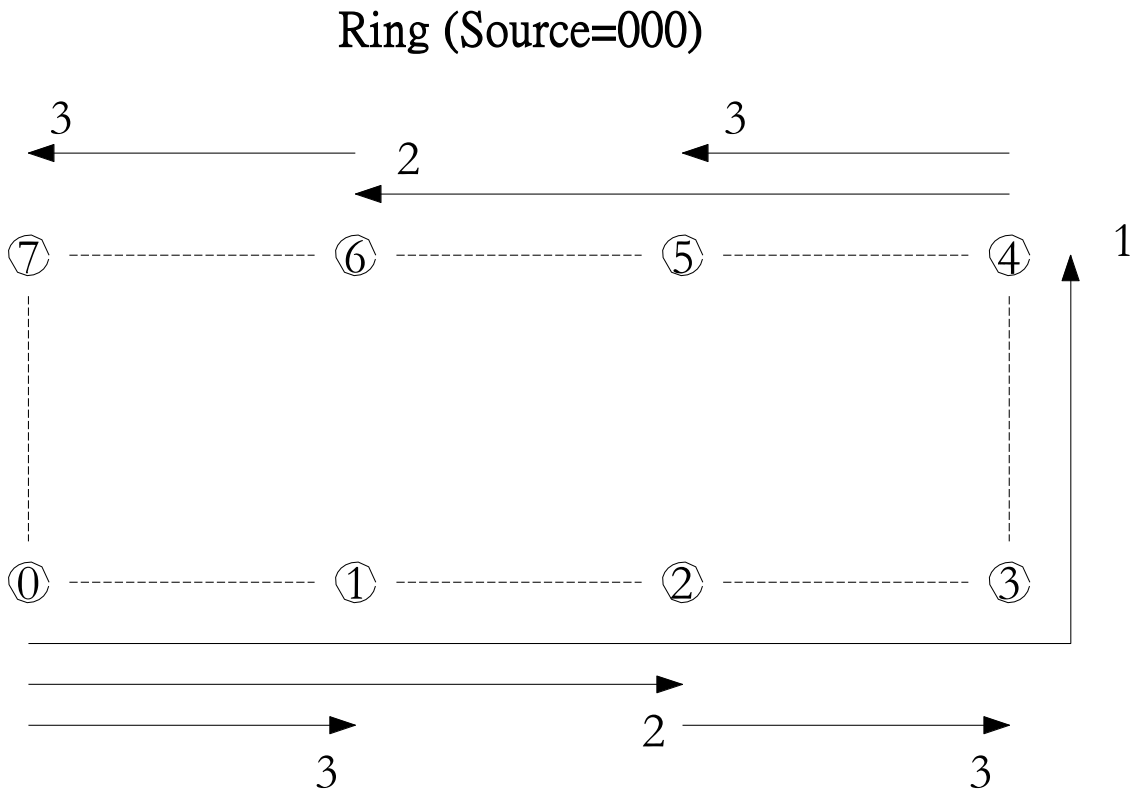
Input:

- The message M is stored locally on the root

Output:

- The message M is stored locally on all processes

One-to-all broadcast

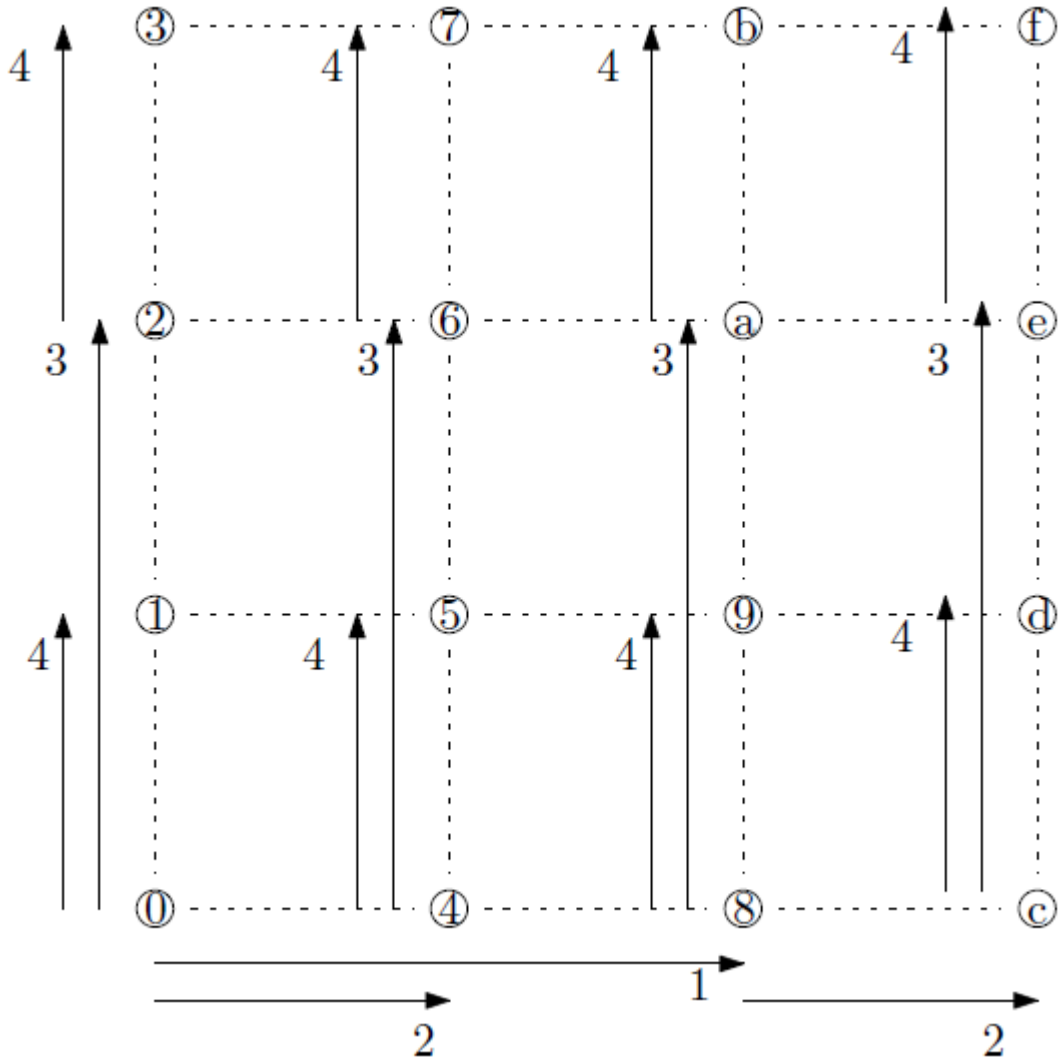


- Recursive doubling
- Double the number of active processes in each step

One-to-all broadcast

Mesh (Source=0000)

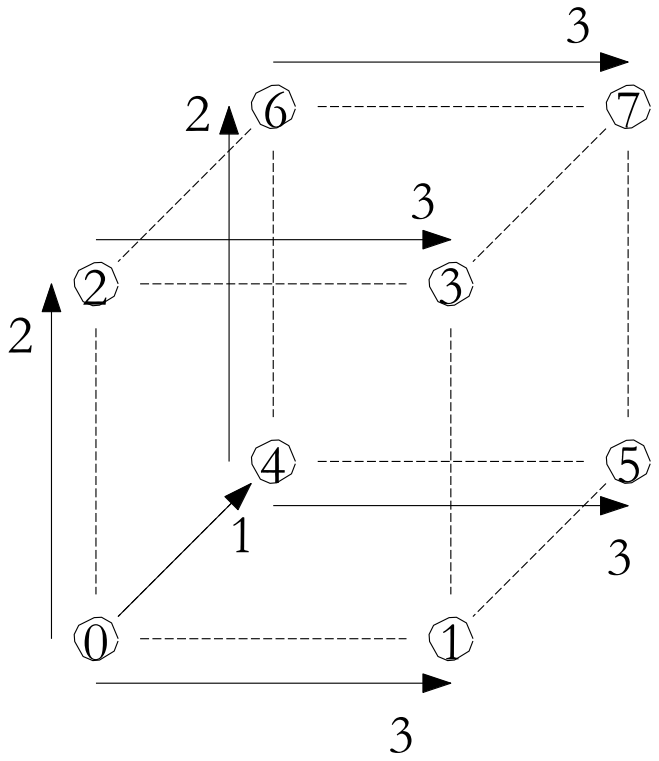
- Use ring algorithm on the root's mesh row
- Use ring algorithm on all mesh columns in parallel



One-to-all broadcast

Hypercube (Source=000)

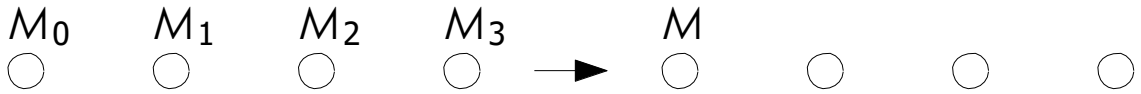
- Generalize mesh algorithm to d dimensions



One-to-all broadcast

- Number of steps: $d = \log_2 p$
- Time per step: $t_s + t_w m$
- Total time: $(t_s + t_w m) \log_2 p$
- In particular, note that broadcasting to p^2 processes is only twice as expensive as broadcasting to p processes ($\log_2 p^2 = 2 \log_2 p$)

All-to-one reduction



$$M := M_0 \oplus M_1 \oplus M_2 \oplus M_3$$

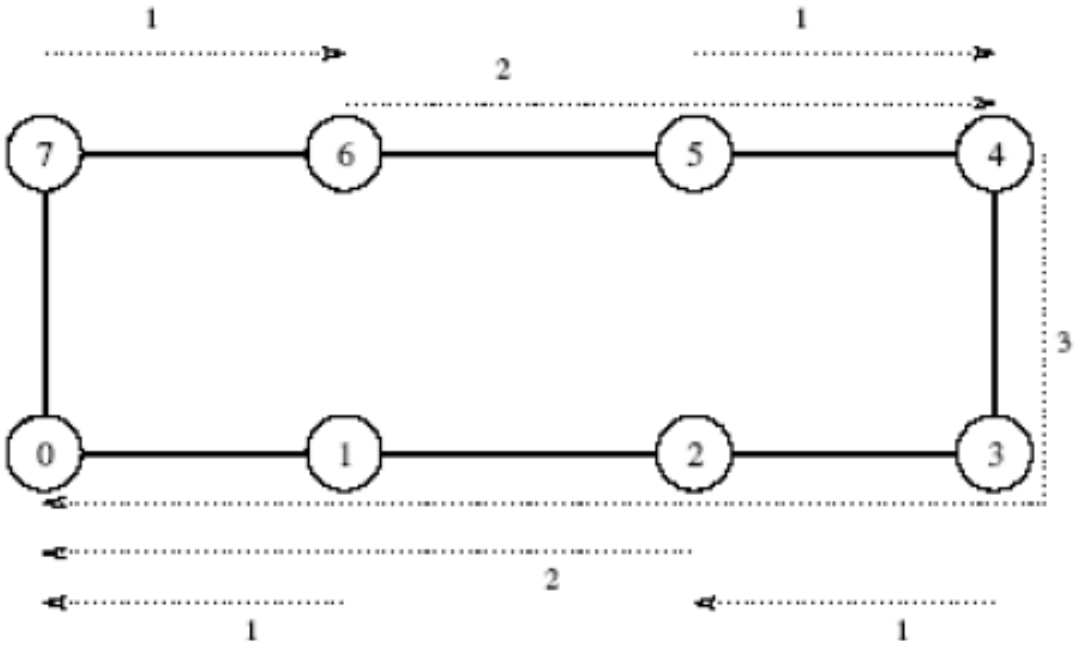
Input:

- The p messages M_k for $k = 0, 1, \ldots, p-1$
- The message M_k is stored locally on process k
- An associative reduction operator $\oplus$
- E.g., $\oplus \in \{+, \times, \max, \min\}$

Output:

- The "sum" $M := M_0 \oplus M_1 \oplus \cdots \oplus M_{p-1}$ stored locally on the root

All-to-One Reduction on Rings (2/2)



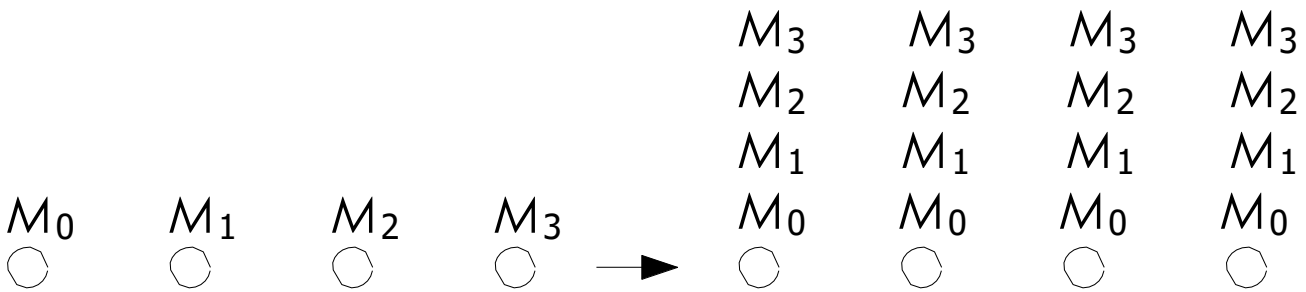
Reduction on an eight-node ring with node 0 as the destination of the reduction.

All-to-one reduction

Algorithm

- Analogous to one-to-all broadcast algorithm
- Analogous time (plus the time to compute $a \oplus b$)
- Reverse order of communications
- Reverse direction of communications
- Combine incoming message with local message using $\oplus$

All-to-all broadcast



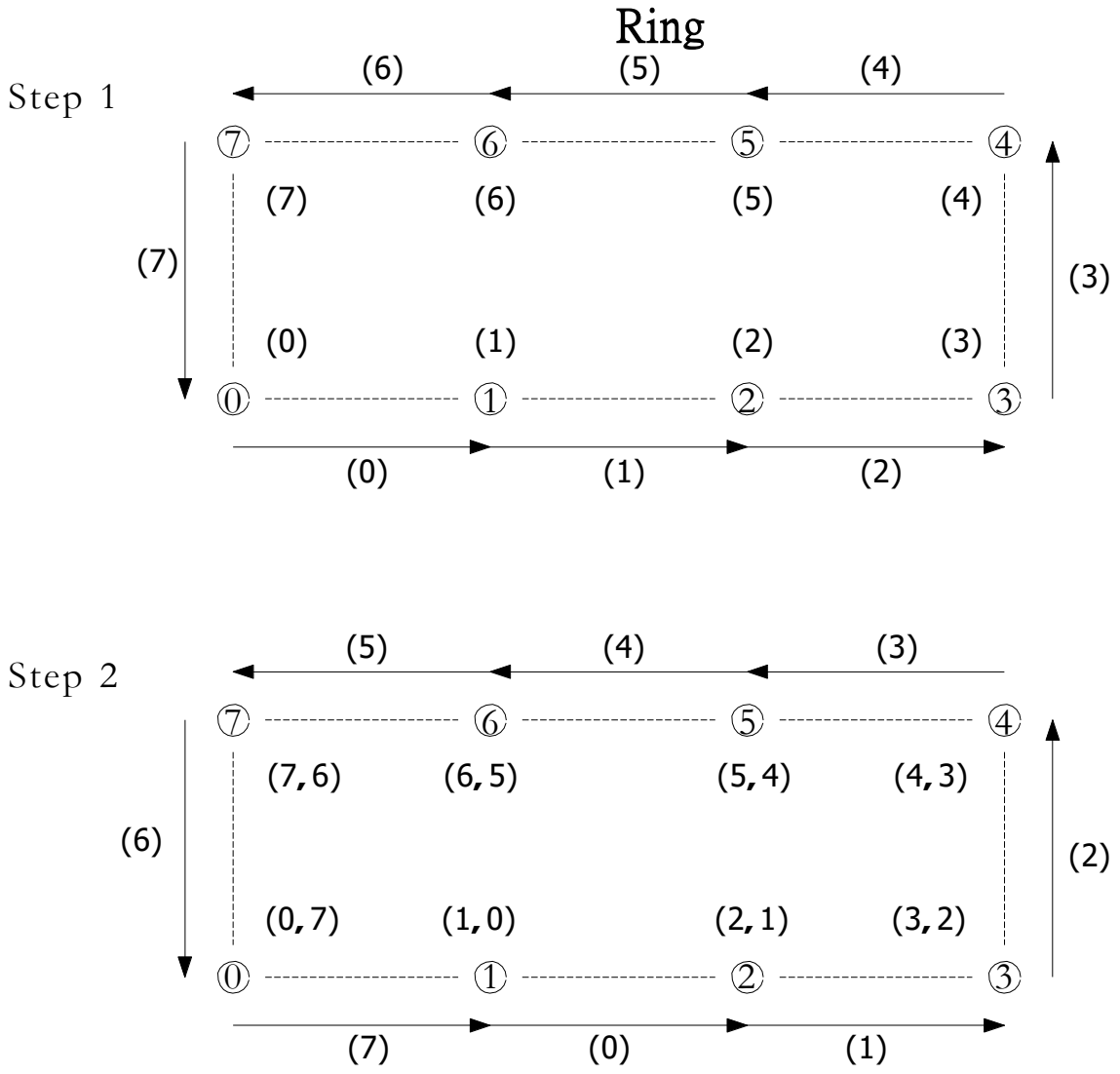
Input:

- The p messages M_k for $k = 0, 1, \ldots, p-1$
- The message M_k is stored locally on process k

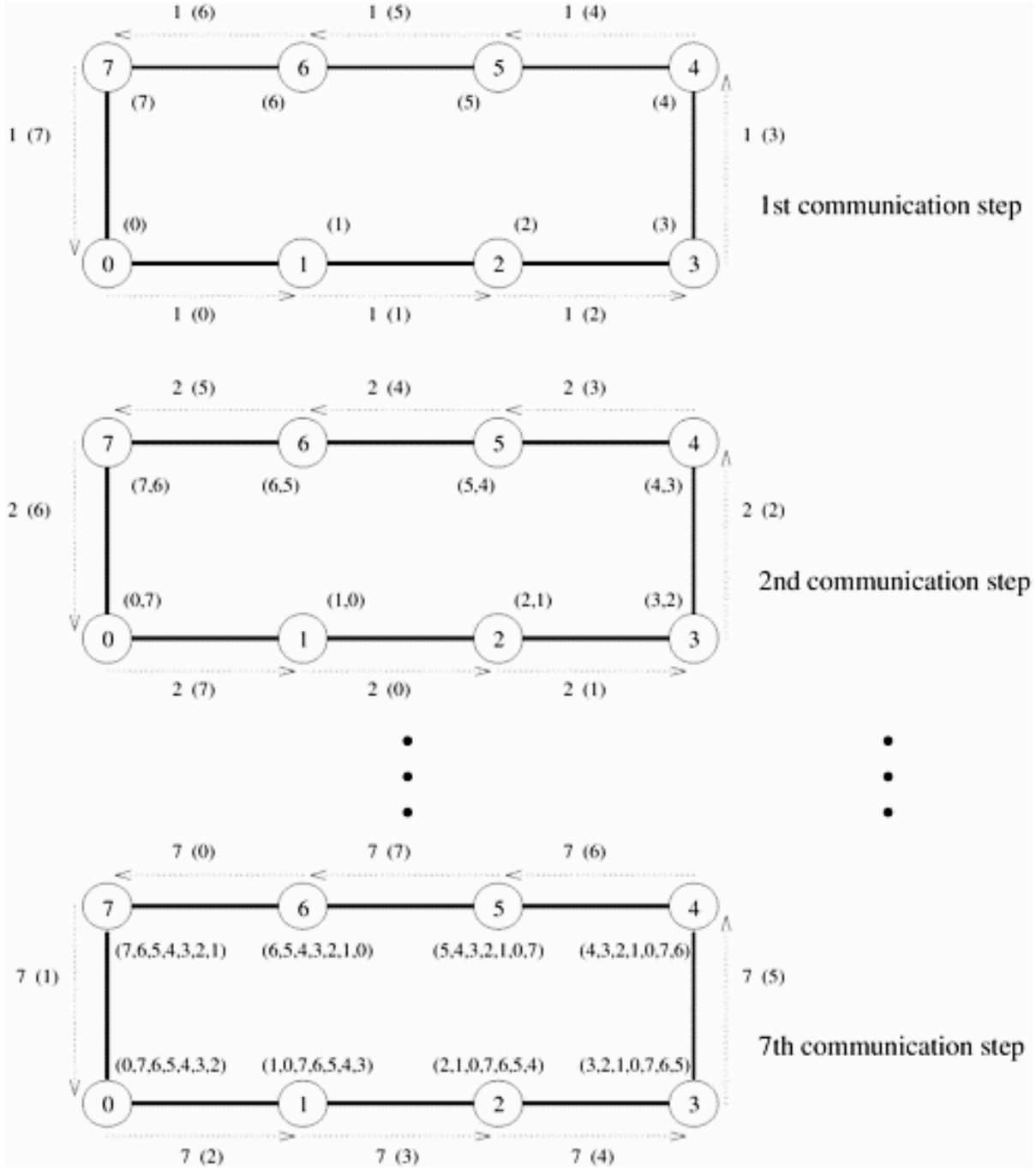
Output:

- The p messages M_k for $k = 0, 1, \ldots, p-1$ are stored locally on all processes

All-to-all broadcast



and so on...

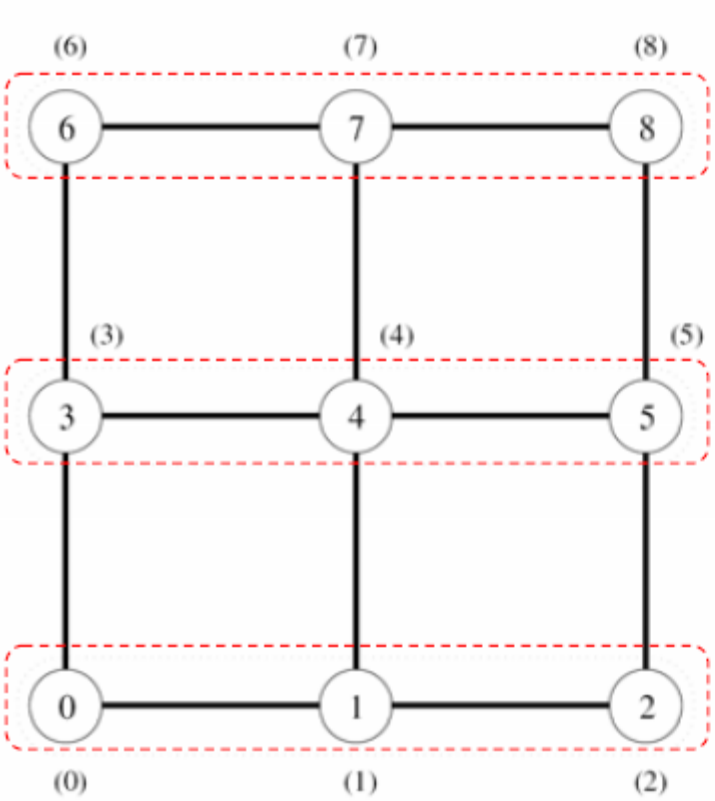

1 (6)
1 (5)
1 (4)
7
6
5
4
(7)
(6)
(5)
(4)
1 (7)
1 (3)
(0)
(1)
(2)
(3)
0
1
2
3
1 (0)
1 (1)
1 (2)
1st communication step
2 (5)
2 (4)
2 (3)
(7,6)
(6,5)
(5,4)
(4,3)
2 (6)
2 (2)
(0,7)
(1,0)
(2,1)
(3,2)
2 (7)
2 (0)
2 (1)
2nd communication step
7 (0)
7 (7)
7 (6)
(7,6,5,4,3,2,1)
(6,5,4,3,2,1,0)
(5,4,3,2,1,0,7)
(4,3,2,1,0,7,6)
7 (1)
7 (5)
(0,7,6,5,4,3,2)
(1,0,7,6,5,4,3)
(2,1,0,7,6,5,4)
(3,2,1,0,7,6,5)
7 (2)
7 (3)
7 (4)
7th communication step

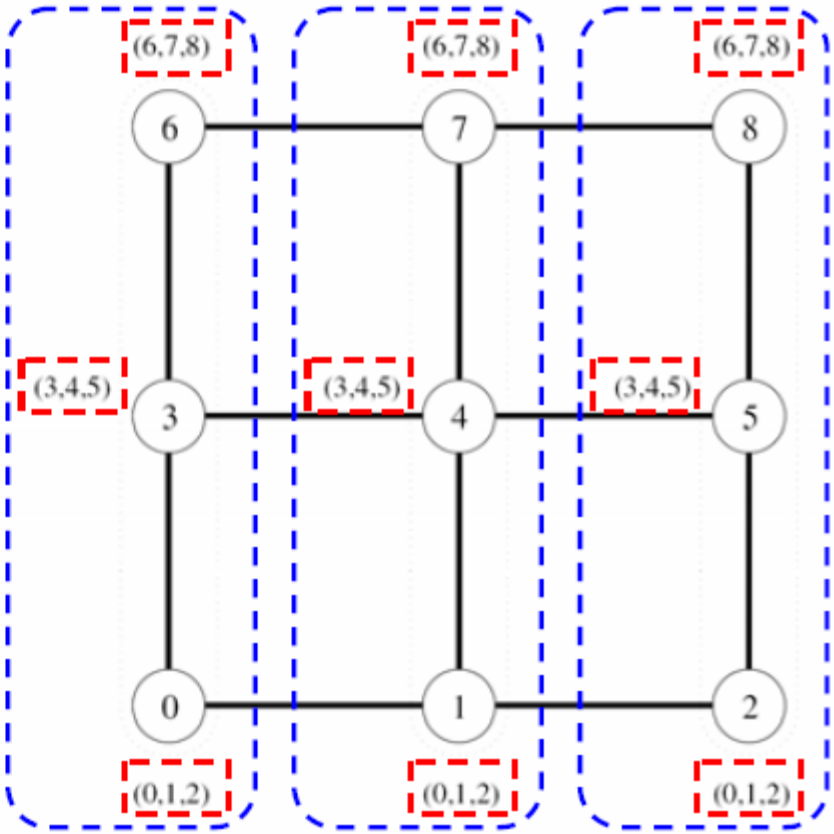
All-to-all broadcast

Mesh algorithm

The mesh algorithm is based on the ring algorithm:

- Apply the ring algorithm to all mesh rows in parallel
- Apply the ring algorithm to all mesh columns in parallel





All-to-all broadcast

Time of mesh algorithm

(Assuming a $\sqrt{p} \times \sqrt{p}$ mesh for simplicity)

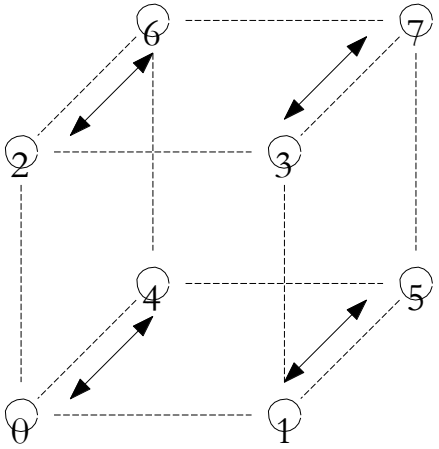
- Apply the **ring** algorithm to all mesh rows in parallel
  - Number of steps: $\sqrt{p} - 1$
  - Time per step: $t_s + t_w m$
  - Total time: $(\sqrt{p} - 1)(t_s + t_w m)$
- Apply the **ring** algorithm to all mesh columns in parallel
  - Number of steps: $\sqrt{p} - 1$
  - Time per step: $t_s + t_w \sqrt{p} m$
  - Total time: $(\sqrt{p} - 1)(t_s + t_w \sqrt{p} m)$
- Total time: $2(\sqrt{p} - 1)t_s + (p - 1)t_w m$

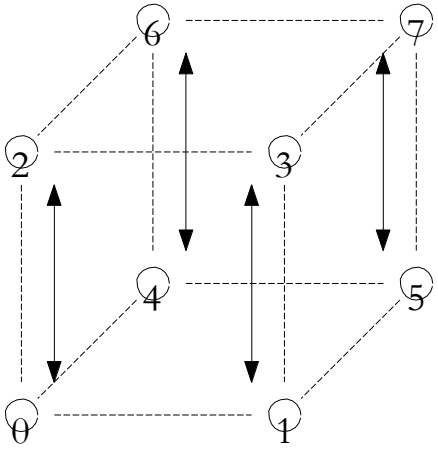
All-to-all broadcast

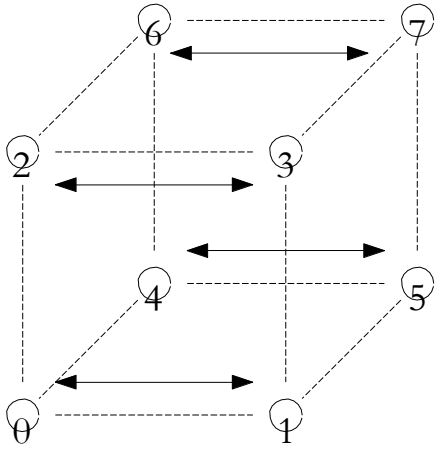
Hypercube algorithm

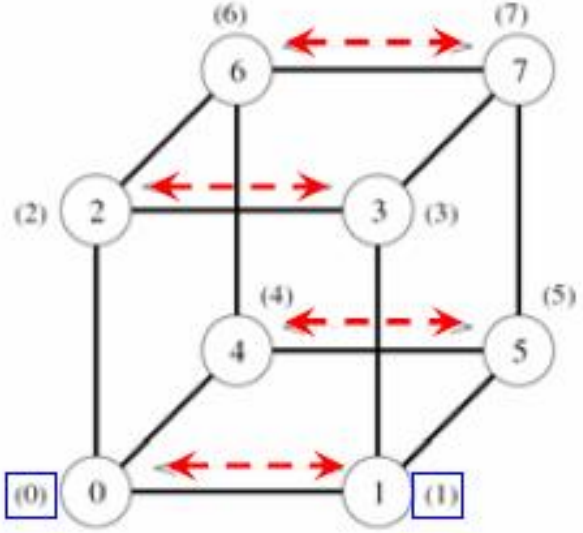
The hypercube algorithm is also based on the ring algorithm:

- For each dimension d of the hypercube in sequence:
- Apply the ring algorithm to the 2^{d-1} links in the current dimension in parallel

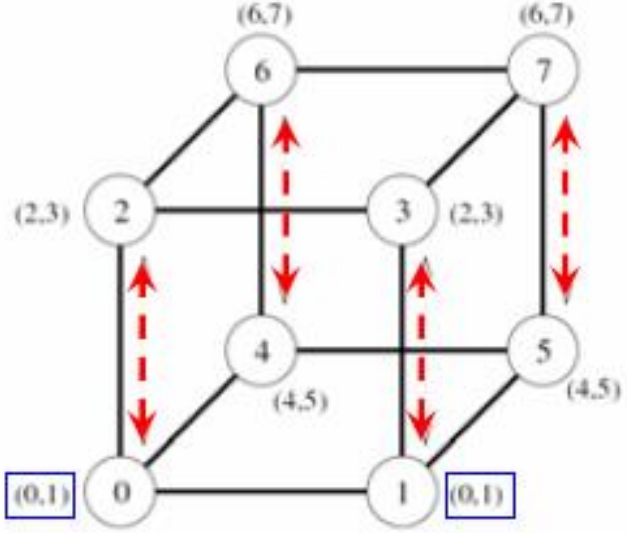




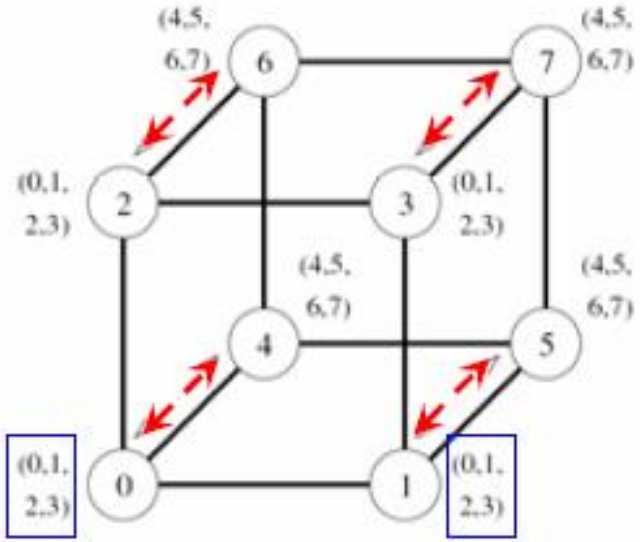




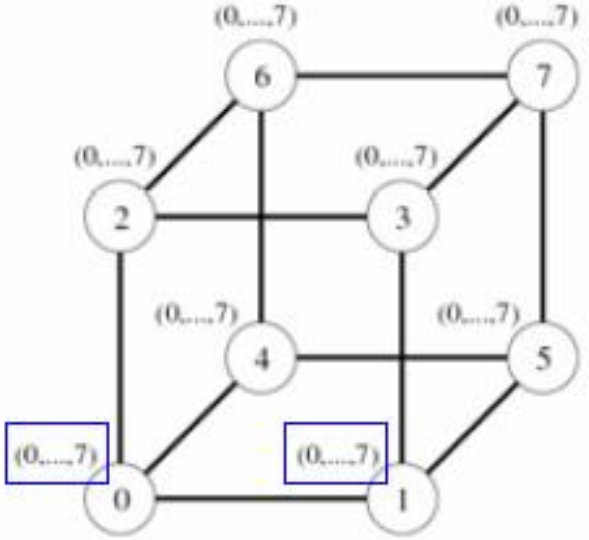
(a) Initial distribution of messages



(b) Distribution before the second step

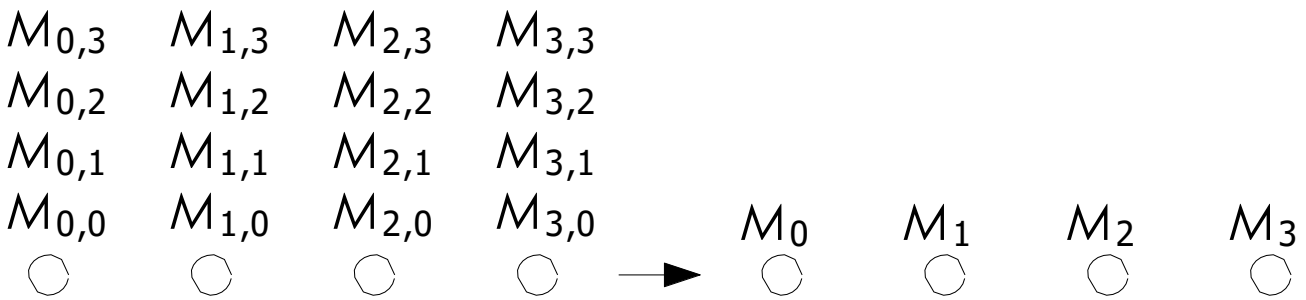


(c) Distribution before the third step



(d) Final distribution of messages

All-to-all reduction



$$M_r := M_{0,r} \oplus M_{1,r} \oplus M_{2,r} \oplus M_{3,r}$$

Input:

- The p^2 messages $M_{r,k}$ for $r, k = 0, 1, \ldots, p-1$
- The message $M_{r,k}$ is stored locally on process r
- An associative reduction operator $\oplus$

Output:

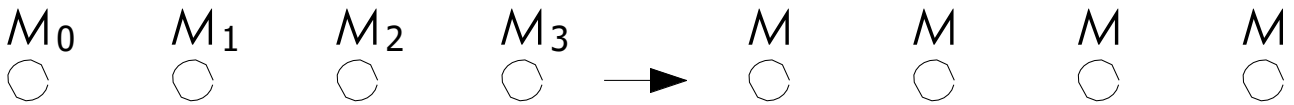
- The "sum" $M_r := M_{0,r} \oplus M_{1,r} \oplus \cdots \oplus M_{p-1,r}$ stored locally on each process r

All-to-all reduction

Algorithm

- Analogous to all-to-all broadcast algorithm
- Analogous time (plus the time for computing $a \oplus b$)
- Reverse order of communications
- Reverse direction of communications
- Combine incoming message with part of local message using $\oplus$

All-reduce



$$M := M_0 \oplus M_1 \oplus M_2 \oplus M_3$$

Input:

- The p messages M_k for $k = 0, 1, \ldots, p-1$
- The message M_k is stored locally on process k
- An associative reduction operator $\oplus$

Output:

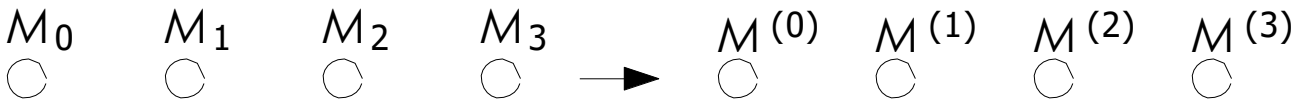
- The "sum" $M := M_0 \oplus M_1 \oplus \cdots \oplus M_{p-1}$ stored locally on all processes

All-reduce

Algorithm

- Analogous to all-to-all broadcast algorithm
- Combine incoming message with local message using $\oplus$
- Cheaper since the message size does not grow
- Total time: $(t_s + t_w m) \log_2 p$

Prefix sum



$$M^{(k)} := M_0 \oplus M_1 \oplus \cdots \oplus M_k$$

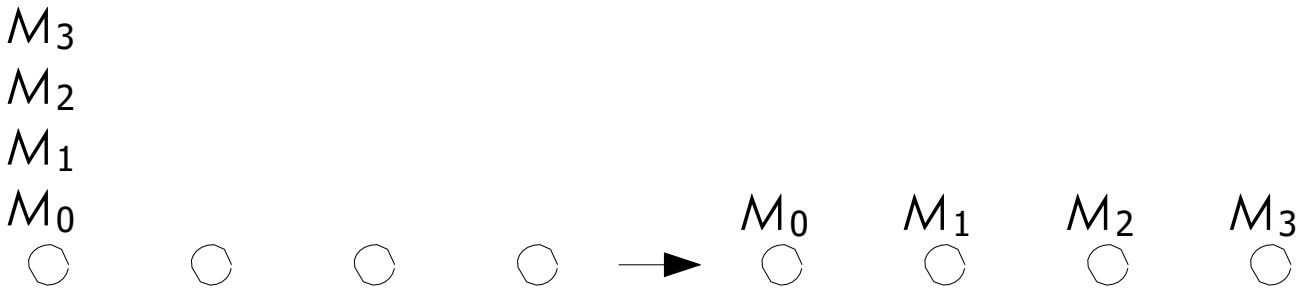
Input:

- The p messages M_k for $k = 0, 1, \ldots, p-1$
- The message M_k is stored locally on process k
- An associative reduction operator $\oplus$

Output:

- The "sum" $M^{(k)} := M_0 \oplus M_1 \oplus \cdots \oplus M_k$ stored locally on process k for all k

Scatter (one-to-all personalized)

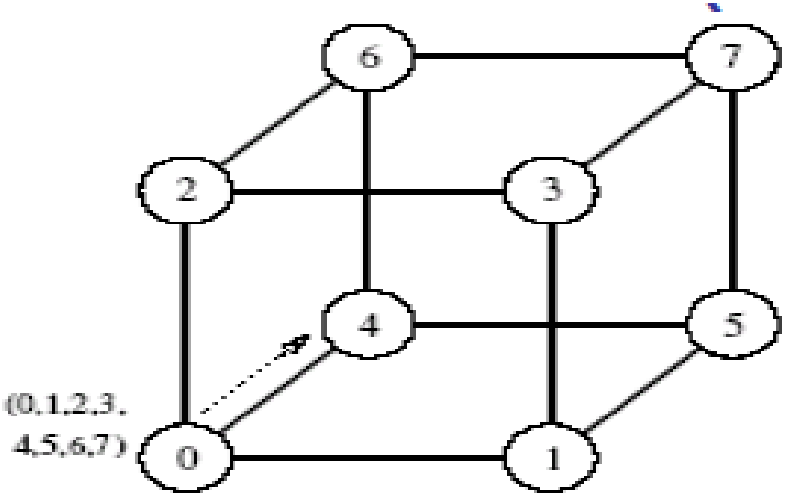


Input:

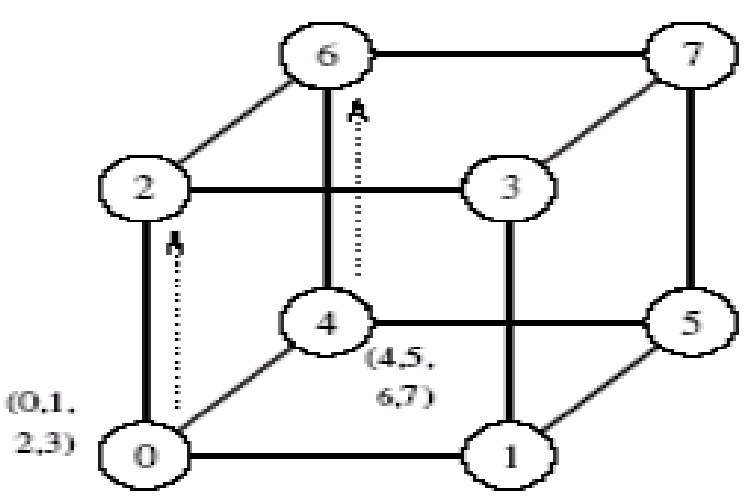
- The p messages M_k for $k = 0, 1, \ldots, p-1$ stored locally on the root

Output:

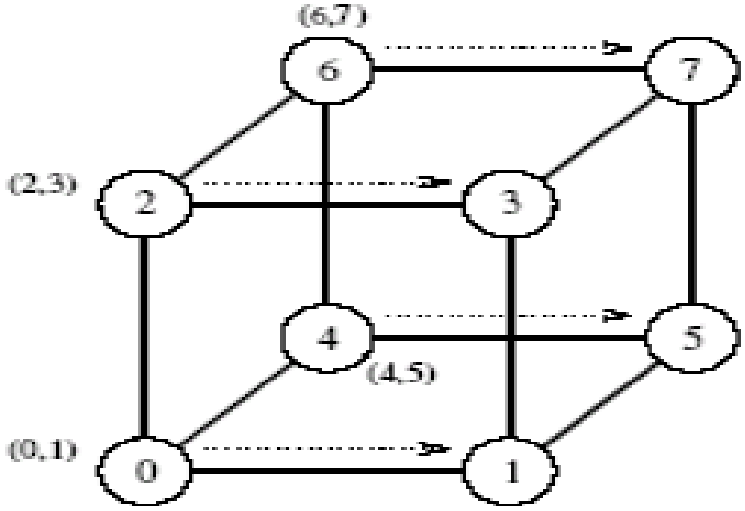
- The message M_k stored locally on process k for all k



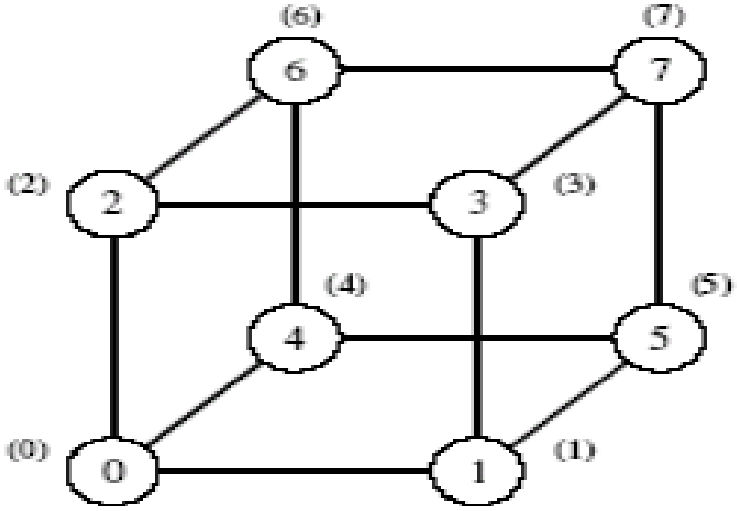
(a) Initial distribution of messages



(b) Distribution before the second step



(c) Distribution before the third step



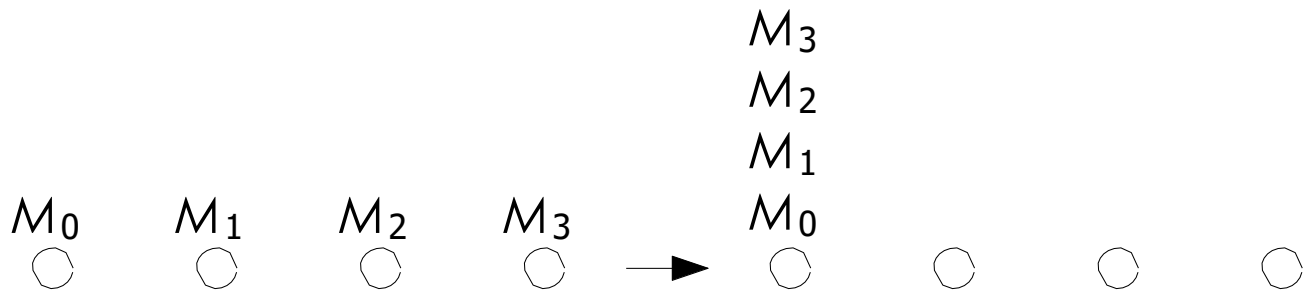
(d) Final distribution of messages

Scatter

Algorithm

- Analogous to one-to-all broadcast algorithm
- Send half of the messages in the first step, send one quarter in the second step, and so on
- More expensive since several messages are sent in each step
- Total time: $t_s \log_2 p + t_w(p-1)m$

Gather



Input:

- The p messages M_k for $k = 0, 1, \ldots, p-1$
- The message M_k is stored locally on process k

Output:

- The p messages M_k stored locally on the root

Gather

Algorithm

- Analogous to scatter algorithm
- Analogous time
- Reverse the order of communications
- Reverse the direction of communications

Summary

Hypercube

Operation	Time
One-to-all broadcast	$(t_s + t_w m) \log_2 p$
All-to-one reduction	$(t_s + t_w m) \log_2 p$
All-reduce	$(t_s + t_w m) \log_2 p$
Prefix sum	$(t_s + t_w m) \log_2 p$
All-to-all broadcast	$t_s \log_2 p + t_w(p-1)m$
All-to-all reduction	$t_s \log_2 p + t_w(p-1)m$
Scatter	$t_s \log_2 p + t_w(p-1)m$
Gather	$t_s \log_2 p + t_w(p-1)m$
All-to-all personalized	$(t_s + t_w m)(p-1)$